

O **PSoft Arena** é uma plataforma para submissão e avaliação de soluções de problemas de projeto de software orientado a objetos.

Na plataforma, o professor cadastra problemas, estratégias de avaliação, turmas, atividades e modelos de LLM. Os estudantes submetem soluções em Java e acompanham o processamento e os resultados das avaliações.

A avaliação é realizada automaticamente, sem a execução do código submetido, por meio de análise estática e de APIs externas de LLM.

User Stories

US1 - Cadastro de Problemas

Eu, como professor, quero cadastrar problemas de projeto de software para disponibilizá-los posteriormente em atividades para minhas turmas.

- Um problema deve possuir, no mínimo:
  - título;
  - descrição;
  - requisitos funcionais;
  - requisitos de projeto;
  - status.
- Um problema deve ser criado com status **Ativo**.
- O professor deve poder consultar e editar problemas.
- O professor poderá desativar um problema, alterando seu status para **Inativo**.
- Problemas inativos não poderão ser associados a novas atividades.
- O professor poderá remover um problema somente quando ele ainda não tiver sido utilizado em nenhuma atividade.
- Problemas que já tenham sido utilizados em atividades não poderão ser removidos.

US2 - Cadastro de Estudantes

Eu, como professor, quero cadastrar estudantes para que eles possam participar das atividades da plataforma.

- Um estudante deve possuir, no mínimo:

    - nome;
    - matrícula;
    - e-mail.
- A matrícula deve identificar unicamente um estudante.
- O professor deve poder criar, consultar e editar estudantes.

US3 - Cadastro de Estratégias de Avaliação

Eu, como professor, quero cadastrar estratégias de avaliação para definir diferentes formas de analisar as soluções submetidas pelos estudantes.

- O sistema deve possuir os seguintes tipos de estratégia:
    - Code Smells;
    - Baixo Acoplamento, Alta Coesão e Princípios (Programe para uma Interface e use Herança com moderação);
    - Aplicação de Padrões de Projeto.
- Uma estratégia deve possuir:
    - nome;
    - tipo;
    - descrição;
    - prompt de avaliação.
- O prompt deve orientar a avaliação realizada pela estratégia.
- O professor poderá definir e alterar o prompt de cada estratégia.
- Podem existir várias estratégias do mesmo tipo, com prompts diferentes.
- Uma mesma estratégia poderá ser utilizada em várias atividades.

US4 - Cadastro de Turmas

Eu, como professor, quero cadastrar turmas para organizar os estudantes que participarão das atividades.

- Uma turma deve possuir, no mínimo:
    - nome;
    - código;
    - período letivo;
    - professor responsável;
    - status.
- Uma turma deve ser criada com status Ativa.
- O professor deve poder consultar e editar suas turmas.
- O professor poderá encerrar uma turma, alterando seu status para Encerrada.
- Turmas encerradas continuarão disponíveis para consulta do histórico.
- Turmas encerradas não poderão receber novas associações de estudantes.
- Turmas encerradas não poderão receber novas atividades.

US5 - Associação de Estudantes às Turmas

Eu, como professor, quero associar estudantes às minhas turmas, para definir quem poderá participar das atividades.

- O professor deve poder adicionar e remover estudantes de uma turma.
- Somente turmas ativas poderão receber novas associações.
- Um estudante pode participar de várias turmas.
- Uma turma pode possuir vários estudantes.
- O sistema não deve permitir a mesma associação entre estudante e turma mais de uma vez.
- Apenas estudantes associados à turma poderão participar das atividades daquela turma.

US6 - Criação de Atividades

Eu, como professor, quero criar atividades para uma turma, para disponibilizar um problema e definir como suas soluções serão avaliadas.

- Uma atividade deve estar associada:
    - a uma turma;
    - a um problema;
    - a uma estratégia de avaliação;
    - a um LLM padrão.
- A turma deve estar ativa no momento da criação da atividade.
- O problema deve estar ativo no momento da criação da atividade.
- Cada atividade deve utilizar uma única estratégia de avaliação.
- O professor deve configurar o LLM padrão da atividade informando:
    - nome do modelo;
    - URL da API (usar APIs públicas https://grok.com/);
    - chave de acesso à API que deve ser armazenada de forma segura.
- LLM padrão será utilizado na avaliação das submissões da atividade.
- Uma atividade deve possuir, no mínimo:
    - título;
    - descrição;
    - prazo para submissão;
    - status.
- Uma atividade deve ser criada com status Aberta.
- Atividades abertas aceitam novas submissões.
- O professor poderá encerrar uma atividade, alterando seu status para Concluída.
- Atividades concluídas não aceitam novas submissões.

US7 - Consulta de Atividades

Eu, como estudante, quero consultar as atividades das turmas das quais faço parte, para saber quais problemas devo resolver.

- O estudante deve visualizar apenas atividades associadas às turmas das quais participa.
- O estudante deve poder consultar:
    - título da atividade;
    - turma;
    - problema;
    - descrição;
    - requisitos funcionais;
    - requisitos de projeto;
    - estratégia de avaliação;
    - prazo;
    - status.
- O estudante não deve ter acesso às atividades de turmas das quais não participa.
- Atividades concluídas podem continuar sendo consultadas, mas não aceitam novas submissões.

US8 - Submissão de Soluções

Eu, como estudante, quero submeter minha solução para uma atividade, para que ela seja avaliada pela plataforma.

- A solução deve ser submetida em Java.
- Uma submissão deve estar associada:
    - ao estudante;
    - à atividade;
    - à data e hora da submissão;
    - ao código-fonte enviado.
- O estudante pode realizar mais de uma submissão para a mesma atividade.
- Cada nova submissão deve ser registrada como uma nova versão da solução.
- Somente estudantes pertencentes à turma da atividade podem realizar submissões.
- Somente atividades abertas aceitam novas submissões.
- Após o envio, a submissão deve ser registrada com status Submetida.
- O sistema deve informar ao estudante que a submissão foi recebida.

US9 - Acompanhamento da Submissão

Eu, como estudante, quero acompanhar o andamento da minha submissão, para saber em que etapa da avaliação ela se encontra.

- Uma submissão deve possuir os seguintes estados:
    - Submetida;
    - Em fila;
    - Processando;
    - Concluída;
    - Falha.
- Uma nova submissão deve iniciar no estado Submetida.
- Após ser encaminhada para processamento, a submissão deve passar para Em fila.

- Quando o processamento for iniciado, a submissão deve passar para Processando.
- Ao finalizar a avaliação, a submissão deve passar para:
    - Concluída, quando a avaliação for realizada com sucesso; ou
    - Falha, quando ocorrer um erro que impeça sua conclusão.
- O estudante deve poder consultar o estado atual de suas submissões.
- O sistema deve registrar o histórico de mudanças de estado.
- Toda mudança de estado deve gerar uma notificação ao estudante (print no terminal).
- O processamento das submissões deve ocorrer de forma assíncrona.
- As submissões devem ser processadas por meio de uma fila centralizada.

US10 - Avaliação da Solução

Eu, como professor, quero que as soluções submetidas sejam avaliadas automaticamente, para verificar sua adequação ao problema proposto.

- Cada submissão deve possuir uma única avaliação final.
- A avaliação deve considerar:
    - os requisitos funcionais definidos para o problema;
    - os requisitos de projeto definidos para o problema;
    - a estratégia de avaliação associada à atividade.
- A avaliação deve ser realizada por meio de análise estática do código-fonte, sem executar a solução submetida.
- A avaliação deve utilizar o LLM padrão configurado para a atividade.
- Quando a atividade possuir múltiplos LLMs configurados, todos os modelos devem avaliar a mesma submissão, utilizando as mesmas informações do problema, da solução e da estratégia de avaliação.
- A avaliação deve utilizar o prompt definido na estratégia de avaliação.
- Quando forem utilizados múltiplos LLMs:
    - o sistema deve registrar o resultado produzido por cada modelo;
    - os resultados dos modelos devem ser considerados na composição da avaliação final;
    - todos os modelos devem produzir resultados válidos para que a avaliação seja concluída.
- Uma avaliação concluída deve produzir uma nota de 0 a 10.
- O resultado da avaliação deve apresentar:
    - nota;
    - evidências que sustentem a avaliação;
    - justificativa da nota.
- A avaliação deve resultar em Falha quando não for possível concluí-la, incluindo situações em que:
    - um dos modelos de LLM não responder;
    - a API de um dos modelos retornar erro;
    - ocorrer uma falha de comunicação com um dos serviços de LLM;
    - ocorrer qualquer outro erro que impeça a produção de um resultado válido por um dos modelos.

US11 - Consulta do Resultado da Avaliação

Eu, como estudante, quero consultar o resultado da avaliação da minha submissão, para compreender como minha solução foi avaliada.

- O estudante deve visualizar somente os resultados de suas próprias submissões.
- O resultado deve apresentar:
    - nota;
    - justificativa;
    - evidências utilizadas na avaliação;
    - situação da avaliação.
- O resultado deve estar associado à submissão correspondente.
- O resultado somente deve ser disponibilizado após a conclusão da avaliação.
- O estudante deve ser informado quando o resultado estiver disponível.

US12 - Reprocessamento de Submissões

Eu, como professor, quero reprocessar uma submissão que apresentou falha, para que uma nova tentativa de avaliação possa ser realizada.

- Somente submissões com status Falha poderão ser reprocessadas.
- O reprocessamento deve utilizar o mesmo código-fonte originalmente submetido.
- O reprocessamento não deve criar uma nova submissão do estudante.
- Ao ser reprocessada, a submissão deve retornar ao estado Em fila.
- O sistema deve preservar o histórico das tentativas de processamento.
- As novas mudanças de estado decorrentes do reprocessamento devem gerar as respectivas notificações ao estudante.

US13 - Histórico de Submissões

Eu, como estudante, quero consultar meu histórico de submissões, para acompanhar as soluções que já enviei.

- O histórico deve apresentar, no mínimo:
    - atividade;
    - turma;
    - data e hora;
    - versão;
    - status;
    - nota, quando disponível.
- O estudante deve visualizar somente suas próprias submissões.
- O histórico não deve ser perdido quando uma nova versão for submetida.
- Deve ser possível consultar os detalhes de uma submissão específica.

US14 - Consulta das Submissões de uma Atividade

Eu, como professor, quero consultar as submissões de uma atividade, para acompanhar a participação e o andamento dos estudantes.

- O professor deve visualizar as submissões dos estudantes da turma associada à atividade.
- A consulta deve apresentar, no mínimo:
    - estudante;
    - data e hora;
    - versão;
    - status;
    - nota, quando disponível.
- Deve ser possível filtrar as submissões por status.
- Deve ser possível consultar os detalhes de uma submissão e seu resultado, quando disponível.
- O professor não deve visualizar submissões de atividades de turmas pelas quais não é responsável.

US15 - Desempenho em uma Atividade

Eu, como professor, quero consultar o desempenho dos estudantes em uma atividade, para analisar os resultados obtidos pela turma.

- Somente atividades concluídas poderão ter seu desempenho consultado.
- O sistema deve apresentar informações como:
    - quantidade de estudantes;
    - quantidade de estudantes que realizaram submissões;
    - quantidade de avaliações concluídas;
    - média das notas;
    - maior nota;
    - menor nota.
- Deve ser possível consultar o desempenho individual dos estudantes na atividade.
- O professor deve poder consultar a nota e o resultado da avaliação de cada estudante.

US16 - Desempenho do Estudante na Turma

Eu, como estudante, quero consultar meu desempenho em uma turma, para acompanhar minha evolução ao longo das atividades.

- O sistema deve apresentar as atividades da turma e a situação do estudante em cada uma delas.
- Para cada atividade, deve ser possível visualizar:
    - quantidade de submissões;
    - situação da última submissão;
    - nota obtida, quando disponível.
- O sistema deve apresentar indicadores gerais do estudante na turma, como:
    - média das notas;
    - maior nota;
    - menor nota;
    - quantidade de atividades avaliadas.
- O estudante somente poderá consultar seu próprio desempenho.

US17 - Consulta de Ranking

Eu, como usuário, quero consultar o ranking de uma atividade ou turma, para acompanhar e comparar o desempenho dos estudantes.

- Estudantes podem consultar os rankings das turmas das quais fazem parte.
- Professores podem consultar os rankings das turmas pelas quais são responsáveis.
- O ranking de uma atividade deve considerar os estudantes participantes da atividade.
- O ranking da turma deve considerar o desempenho dos estudantes nas atividades da turma.
- O ranking deve apresentar, no mínimo:
    - posição;
    - estudante;
    - pontuação.
- O sistema deve tratar empates de forma consistente.

US18 - Cadastro e Configuração de Modelos de LLM

Eu, como professor, quero cadastrar e configurar diferentes modelos de LLM, para utilizar mais de um modelo na avaliação das soluções.

- O professor poderá cadastrar um modelo de LLM informando:
    - nome do modelo;
    - URL da API;
    - chave de acesso à API.
- O professor poderá consultar, editar e remover modelos de LLM cadastrados.
- A chave de acesso à API não deve ser apresentada nas consultas realizadas pelo sistema.
- Um modelo de LLM cadastrado poderá ser utilizado em várias atividades.
- O professor poderá adicionar modelos de LLM adicionais à configuração de uma atividade.
- Uma atividade poderá utilizar um ou mais modelos de LLM, mantendo o LLM padrão definido na criação da atividade.
- O LLM configurado inicialmente na atividade como padrão deverá permanecer identificado como LLM padrão.
- Os modelos adicionais deverão ser selecionados entre os modelos de LLM cadastrados no sistema.
- O professor poderá remover um modelo adicional da configuração da atividade, sem remover o modelo de LLM do cadastro do sistema.
- Quando uma atividade utilizar múltiplos modelos, todos os modelos configurados deverão avaliar a mesma submissão.
- O sistema deverá registrar separadamente o resultado produzido por cada modelo utilizado na avaliação.
- O sistema deverá registrar eventuais falhas ocorridas durante a avaliação por cada modelo, incluindo ausência de resposta, erro da API ou falha de comunicação.
- A falha de qualquer um dos modelos configurados deverá impedir a conclusão da avaliação da submissão.

- Os resultados válidos produzidos pelos modelos deverão ser considerados na composição de uma única avaliação final para a submissão.

US 19 - Feedback sobre a Avaliação

Eu, como estudante, quero enviar um feedback sobre o resultado da avaliação da minha submissão, para informar minha percepção sobre a avaliação realizada.

- O estudante poderá enviar feedback após a avaliação da submissão ser concluída.
- O feedback poderá conter, no mínimo:
    - comentário;
    - indicação de concordância ou discordância com o resultado.
- O feedback deve estar associado à avaliação correspondente.
- O estudante poderá consultar o feedback que enviou.
- O professor poderá consultar os feedbacks enviados pelos estudantes de suas turmas.
- O sistema deve manter o histórico dos feedbacks registrados.

US20 - Exportação de Resultados

Eu, como professor, quero exportar os resultados das atividades em formato CSV, para analisar e utilizar os dados fora da plataforma.

- O professor poderá exportar os resultados de uma atividade.
- O professor poderá exportar os resultados de uma turma.
- O arquivo CSV deve conter, no mínimo:
    - estudante;
    - matrícula;
    - atividade;
    - quantidade de submissões;
    - última submissão;
    - status;
    - nota.
- O professor deve poder selecionar a atividade ou turma cujos resultados deseja exportar.
- O arquivo deve ser gerado em formato CSV.

Grupos

- Os grupos devem ser formados por **até 5 pessoas**.

- É obrigatória a colaboração via **GitHub**, sendo apenas um repositório por grupo. O histórico de *commits* no repositório será utilizado durante a avaliação da participação de cada integrante.

- Todo o código-fonte e os testes desenvolvidos devem estar disponíveis no repositório do GitHub, que deve ser gerado por cada grupo em formato **privado** e compartilhado com os **professores** e **monitores** da disciplina.

Formato das Entregas

- Link para o repositório no GitHub com todo o **código-fonte** e **os testes automáticos** desenvolvidos.
- Modelagem em diagrama de classes com os relacionamentos entre as principais classes do sistema (não considerar classes de controller, service ou repository).
- Um documento descrevendo as ***user stories*** implementadas no projeto, a **participação de cada integrante** do grupo nessa implementação (tanto o percentual quanto a descrição da participação, ou seja, informar quem desenvolveu cada US) e, por fim, as **principais decisões de design** tomadas durante seu desenvolvimento (como estilos arquiteturais e/ou padrões de projeto adotados).
- **Apresentação do projeto desenvolvido,** com demonstração das principais funcionalidades implementadas.

Avaliação

- Serão avaliadas a **quantidade**, a **complexidade** e a **corretude** das *user stories* implementadas.
- Serão avaliadas as **decisões de design** tomadas e implementadas durante o desenvolvimento. Por exemplo, o uso adequado de arquiteturas e padrões de projeto.
- Será avaliada a **qualidade do código** desenvolvido. Por exemplo, o uso de princípios de projeto, a atribuição de responsabilidades e a presença de *bad smells* no código.
- Serão avaliadas a **cobertura** e a **qualidade dos testes** automáticos desenvolvidos (abordagem funcional).

Estrutura do sistema

- O sistema web a ser desenvolvido deverá utilizar o ***framework*** **Spring Boot** e deve seguir os **padrões *controller*, *service* e *repository***.
- O código-fonte deve estar **organizado em pacotes**, de modo que se facilite a

diferenciação das classes e de suas respectivas responsabilidades.

- Apesar de **não possuir um *frontend* web**, todas as funcionalidades do sistema devem ser acessíveis via API REST e estar **documentadas através do Swagger**.

Calendário de Entregas

A realização do projeto será realizada em quatro entregas (milestones) seguindo o seguinte cronograma de entregas:

- 1ª Etapa: USs 1 a 5 (03/11/2026) - vale 10% da nota do projeto
- 2ª Etapa: USs 6 a 10 (18/11/2026) - vale 30% da nota do projeto
- 3ª Etapa: USs 11 a 15 (01/12/2026) - vale 20% da nota do projeto
- 4ª Etapa: USs 16 a 20 (15/12//2026) - vale 30% da nota do projeto
- 5ª Etapa: Melhorias (12/02/2027) - vale 10% da nota do projeto